

Introducción

Comunicaciones Dedicated Short Range Communications (DSRC)
Comunicaciones V2X
Elementos de una red vehicular

Veins Simulator

Módulos TraCIDemo11p.cc y TraCIDemo11RSUP.cc

Desarrollo

Parte I: Mapa de SUMO en Veins
Parte II: Modifique el escenario de ejemplo de Veins
Parte III: Ajuste la ubicación del RSU (access point) de su mapa de Veins
Parte IV: Reto de comunicaciones V2V

EV charging
service

Emergency
message

11. Veins: Comunicaciones V2V / V2I

En esta práctica el estudiante explora las comunicaciones vehicle to vehicle V2V y vehicle to infrastructure V2I.

Objetivos

- Conocer los componentes de una red vehicular.
- Identificar los componentes de una red V2X dentro del simulador de comunicaciones DSRC basadas en WAVE.
- Generar una comunicación V2V.
- Generar una comunicación V2I.

11.1 Introducción

En esta práctica, nos centramos en **Veins** (The vehicular network simulation framework), que se posiciona como una herramienta de referencia al sustentar simulaciones de comunicaciones vehiculares en modelos de movilidad vehicular de alta fidelidad, desarrollados y validados por la comunidad científica de transporte y tráfico. En el ámbito de las comunicaciones, Veins destaca por la implementación de la capa física **IEEE 802.11p** y capas superiores definidas en la familia de estándares **IEEE 1609.4 DSRC/WAVE**. Estas incluyen características críticas como la operación multicanal, el acceso al canal con calidad de servicio (*QoS*), y el modelado realista de efectos de ruido e interferencias (modelos de canal), garantizando que el análisis de la red vehicular sea tan preciso como el de la movilidad física.

11.1.1 Comunicaciones Dedicated Short Range Communications (DSRC)

Las Comunicaciones de Corto Alcance Dedicadas (DSRC, por sus siglas en inglés) se definen como una tecnología inalámbrica diseñada específicamente para operar en entornos vehiculares. Su propósito fundamental es establecer una red de conectividad robusta en radios de distancia limitada, permitiendo que los elementos del ecosistema vial, como peatones, vehículos, e infraestructura, se comuniquen entre sí de manera eficiente.

El objetivo principal de esta tecnología es facilitar un intercambio de información rápido y confiable. Al minimizar la latencia en la transmisión de datos, el DSRC permite una respuesta casi instantánea, lo cual es crítico para la seguridad y la gestión del tráfico en tiempo real dentro de los

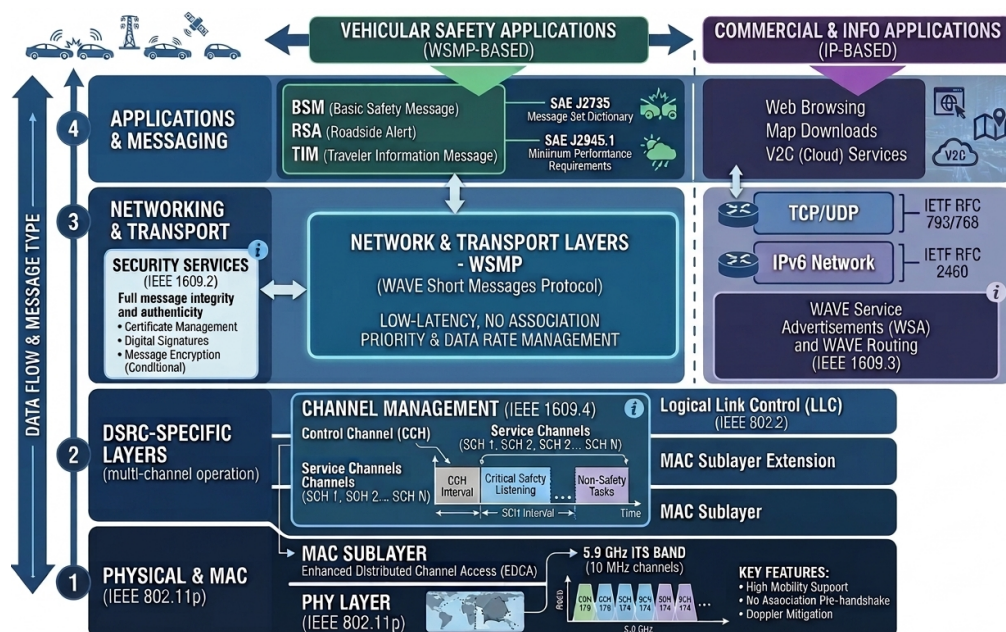


Figure 11.1: Stack de protocolos WAVE/DSRC

Sistemas de Transporte Inteligentes (ITS).

Finalmente, el DSRC habilita dos tipos de interacciones esenciales: la comunicación Vehículo a Vehículo (V2V) y la comunicación Vehículo a Infraestructura (V2I). Esta conectividad permite que los automóviles compartan datos entre ellos y con elementos de la vía (como semáforos o puntos de acceso), mejorando significativamente la coordinación y la seguridad en las carreteras.

La Figura 11.1 ilustra detalladamente la arquitectura de protocolos WAVE/DSRC, la cual se estructura sobre la familia de estándares IEEE 1609 para la gestión de servicios de red, seguridad y operación multicanal. Esta jerarquía técnica opera sobre una capa física basada en el estándar IEEE 802.11p, optimizado para entornos de alta movilidad en la banda de 5.9 GHz.

En los niveles superiores, la arquitectura se bifurca para atender distintos tipos de tráfico: por un lado, las aplicaciones de seguridad utilizan el protocolo de baja latencia WSMP junto con los estándares de mensajería SAE; por otro, las aplicaciones comerciales y de información emplean la pila convencional IPv6/TCP. Finalmente, en los niveles inferiores, el estándar IEEE 1609.4 gestiona el acceso multicanal, garantizando la coexistencia de estos servicios sobre la infraestructura física definida por el estándar 802.11p.

11.1.2 Comunicaciones V2X

En cuanto a las modalidades de interacción, las comunicaciones DSRC permiten el intercambio de datos a través de dos esquemas principales: V2V (Vehículo a Vehículo) y V2I (Vehículo a Infraestructura), ver Fig. 11.2. En el modelo V2V, los vehículos se comunican directamente entre sí para compartir parámetros críticos de seguridad, mientras que en el modelo V2I, los vehículos interactúan con la infraestructura vial para recibir advertencias en tiempo real y datos actualizados sobre el estado del tráfico. Ambos esquemas están diseñados para ofrecer una robustez excepcional

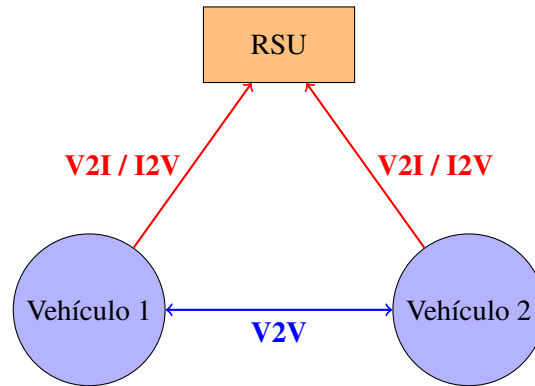


Figure 11.2: Comunicación entre vehículos (V2V) y de vehículos hacia RSU (V2I).

ante la alta movilidad, priorizando mecanismos de acceso al medio rápidos y sencillos que compensen los cambios constantes en la topología de la red debido a las velocidades de desplazamiento de los nodos.

11.1.3 Elementos de una red vehicular

La red se compone de dos elementos de hardware fundamentales: las OBU (On-Board Units) y las RSU (Road Side Units) o access points.

- **OBU:** Las OBU son dispositivos de comunicación instalados en los vehículos que habilitan el intercambio directo de información con otros nodos móviles o fijos.
- **RSU:** Por otro lado, las RSU constituyen la infraestructura física situada a pie de carretera, actuando como puntos de enlace que pueden recibir datos de los vehículos, emitir alertas críticas y distribuir información logística del tráfico para mejorar la seguridad y eficiencia del entorno vehicular.

11.2 Veins Simulator

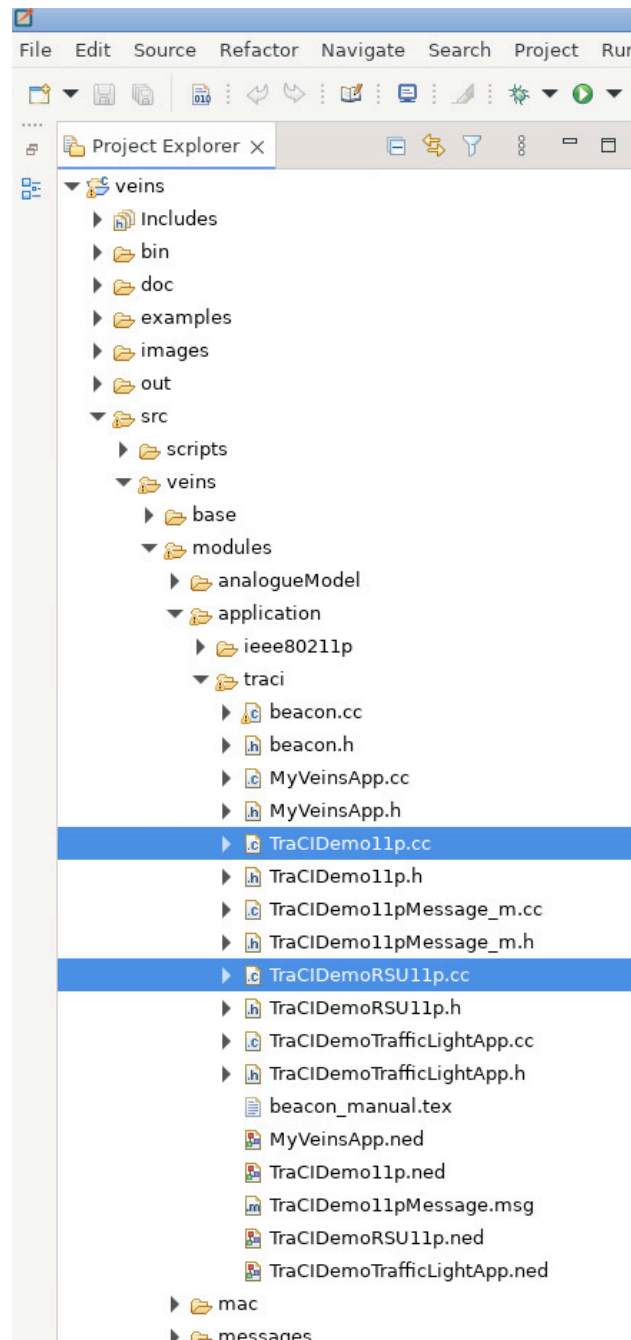
Dentro del entorno de simulación de Veins, a nivel de la capa de aplicación tenemos dos estructuras: (i) `ieee80211p`, y (ii) `traci`, ver Fig. 11.3.

- **ieee80211p:** Esta carpeta contiene implementaciones de referencia para aplicaciones que utilizan directamente el estándar IEEE 802.11p. Incluye archivos que definen cómo se generan y reciben paquetes básicos de datos sobre la interfaz de radio vehicular.
- **traci:** Esta es una de las carpetas más importantes de Veins, ya que gestiona la interfaz TraCI (Traffic Control Interface). Contiene las aplicaciones que permiten la interacción en tiempo real entre la red de comunicaciones (Veins/OMNeT++) y la movilidad del tráfico (SUMO).

11.2.1 Módulos `TraCIDemo11p.cc` y `TraCIDemo11RSUp.cc`

Dos aplicaciones clave en Veins son `TraCIDemo11p.cc` y `TraCIDemo11RSUp.cc`.

- `TraCIDemo11p.cc`: es una implementación de ejemplo incluida en **Veins**. Define el **comportamiento de la aplicación de un vehículo en la simulación**. Es la aplicación de ejemplo



para vehículos. Define qué hace el vehículo cuando recibe un mensaje (por ejemplo, cambiar de ruta o frenar).

- `TraCIDemo11RSUp.cc`: sirve para definir el comportamiento específico de una Road Side Unit (RSU) conocido como el access point. Algunas de sus funciones incluyen (i) procesa los mensajes que recibe de los vehículos (por ejemplo, alertas), (ii) envía mensajes a los vehículos cercanos, ya sea de forma reactiva o periódica, (iii) registra estadísticas como cantidad de mensajes recibidos, retardo, o cobertura.

La Tabla 11.1, muestra de manera resumida las características de estos archivos fundamentales en Veins.

Table 11.1: Comparación entre `TraCIDemo11p` y `TraCIDemo11RSUp`

Aspecto	<code>TraCIDemo11p.cc</code> (Vehículos)	<code>TraCIDemo11RSUp.cc</code> (RSUs)
Tipo de nodo	Nodo móvil (vehículo)	Nodo fijo (unidad carretera)
Movilidad	Controlado por SUMO vía TraCI	No tiene movilidad
Comunicación	Vehículo a vehículo (V2V) y hacia RSUs	Infraestructura a vehículo (I2V)
Función principal	Enviar y recibir mensajes WAVE entre vehículos	Procesar y retransmitir mensajes desde/hacia vehículos
Recepción de mensajes	<code>onWSM()</code> recibe mensajes WSM de otros vehículos o RSUs	<code>onWSM()</code> recibe mensajes enviados por vehículos
Acceso a SUMO	Usa <code>TraCIMobility</code> para leer velocidad, posición, etc.	No usa movilidad propia; posición estática definida en el escenario
Aplicaciones típicas	Difusión de alertas, beacons, advertencias locales	Retransmisión de alertas, servidor de mapas, mensajes de control vial
Ubicación del archivo	<code>veins/modules/application/traci/TraCIDemo11p.cc</code>	<code>veins/modules/application/traci/TraCIDemo11RSUp.cc</code>

11.3 Desarrollo

Valide con el docente

El desarrollo de la práctica se divide en cuatro etapas. Una vez completadas, es obligatoria la validación de los resultados con el docente. **En el informe deberá reportar únicamente el desarrollo de la Parte IV.**

1. **Parte I:** Agregue y ajuste su mapa de SUMO al entorno de Veins.
2. **Parte II:** Navegue en el archivo `TraCIDemo11p.cc` e identifique el escenario de demo de Veins, es decir, el comportamiento de los vehículos en este escenario de ejemplo.
3. **Parte III:** Ajuste la ubicación del RSU de su mapa de Veins.
4. **Parte IV:** Complete el reto propuesto.

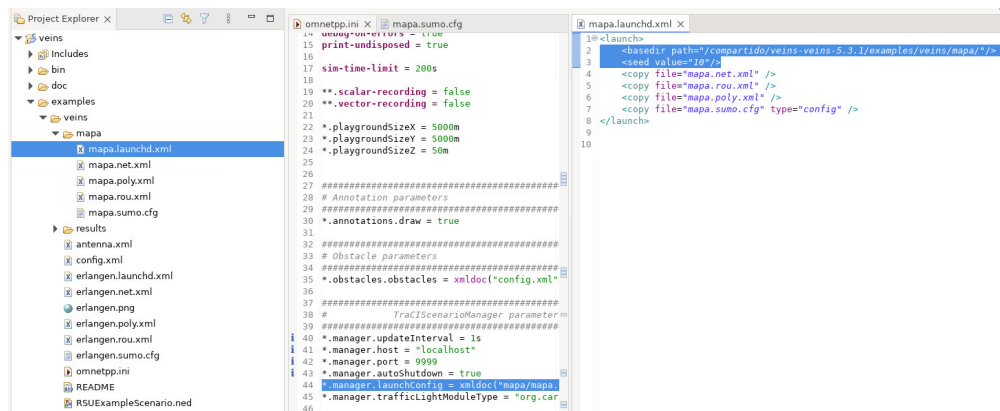


Figure 11.4: Cambios necesarios para aplicar un nuevo mapa al proyecto Veins

11.3.1 Parte I: Mapa de SUMO en Veins

Dentro del proyecto de Veins se sugiere los siguientes cambios en el archivo `omnetpp.ini` (ver Fig. 11.4):

1. Deshabilite las estadísticas por defecto de OMNET++:

```
**.scalar-recording = false
**.vector-recording = false
```

2. Ajuste el tamaño del mapa, de acuerdo a sus archivos de SUMO:

```
*.playgroundSizeX = 2000m
*.playgroundSizeY = 2000m
*.playgroundSizeZ = 50m
```

3. Cree un nuevo directorio dentro de `veins/examples/veins/`, por ejemplo *mapa*, donde almacenaremos los archivos de SUMO (.net, .poly, .sumocfg). Esto es útil dado que si se requieren varios escenarios de prueba se pueden mantener organizados (vea el lado derecho de la Fig.11.4).
4. Dentro del nuevo directorio, cree el archivo `launchd.xml` que contiene las directivas de los archivos de sumo (vea el lado izquierdo de la Fig. 11.4. Adicional a lo definido en el ejemplo `earlangen.launchd.xml`, agregamos dos líneas que definen el nuevo directorio *basedir path* y la semilla para la simulación *seed*:

```
<basedir path="/compartido/veins-veins-5.3.1/examples/veins/mapa/" />
<seed value="10"/>
```

5. Actualice la ubicación del archivo `launchd.xml` dentro del proyecto de Veins.

```
*.manager.launchConfig = xmldoc("mapa/mapa.launchd.xml")
```

6. Ajuste el tiempo de simulación en el `omnetpp.ini`, de acuerdo a su archivo `sumocfg`:

```
sim-time-limit = 200s
```

7. Antes de lanzar la simulación, verifique la ejecución del servidor de SUMO:

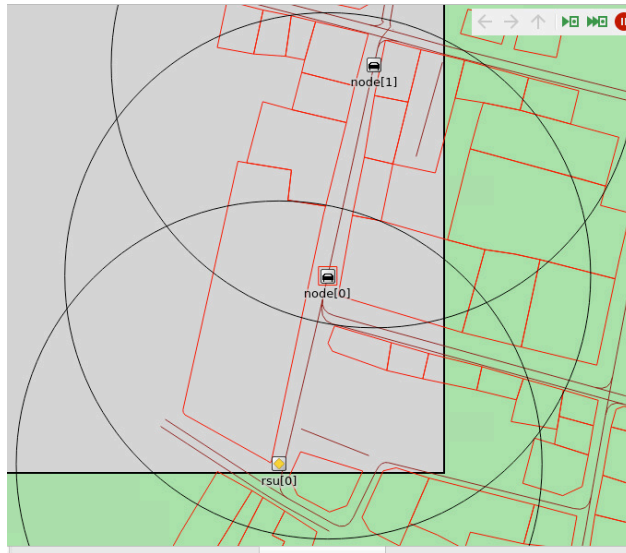


Figure 11.5: Escenario con 2 nodos (OBUs - vehículos) y un access point (RSU)

```
./run_sumo.sh
```

11.3.2 Parte II: Modifique el escenario de ejemplo de Veins

1. Deshabilite el programa de semáforos inteligentes que viene por defecto en Veins. Comente las siguientes líneas del `omnetpp.ini`:

```
#*.manager.trafficLightModuleType = "org.car2x.veins.nodes.TrafficLight"

#*.tls[*].applType = "org.car2x.veins.modules.application.traci.
    TraCIDemoTrafficLightApp"
#*.tls[*].logicType = "org.car2x.veins.modules.world.traci.trafficLight.
    logics.TraCITrafficLightSimpleLogic"
```

2. Dentro del archivo `omnetpp.ini`, localice la configuración `*.node[*].applType = "TraCIDemo11p"`. El valor *TraCIDemo11p* vincula el escenario con el archivo `.ned` que define la topología de red de los vehículos. Por su parte, la lógica operacional del escenario demo se encuentra implementada en el archivo fuente `TraCIDemo11p.cc`, ubicado en la ruta `/src/veins/modules/application/traci/`. **Se requiere realizar un análisis exhaustivo de los métodos descritos dentro de dicho archivo para comprender el comportamiento de la aplicación.**

Los métodos de interés se resumen en la Tabla 11.2 y se listan a continuación:

```
void TraCIDemo11p::onWSM(BaseFrame1609_4* frame)

void TraCIDemo11p::handleSelfMsg(cMessage* msg)

void TraCIDemo11p::handlePositionUpdate(cObject* obj)
```

Table 11.2: Funciones Principales en Veins (TraCIDemo11p)

Función	Disparador (Trigger)	Propósito Principal
onWSM()	Recepción de un mensaje de red (otro vehículo/RSU).	Procesar información inalámbrica entrante y reaccionar ante alertas.
handleSelfMsg()	Temporizador interno (Self-message).	Ejecutar tareas programadas, eventos periódicos o beacons.
handlePositionUpdate()	Movimiento en el simulador de tráfico (SUMO).	Sincronizar la lógica de red con la posición física y ruta del vehículo.

- **onWSM:** se activa automáticamente cada vez que la antena del vehículo detecta un paquete de datos entrante (Wave Short Message) enviado por otro nodo. Su función principal es procesar la información externa recibida, permitiendo que el vehículo reaccione a eventos del entorno, como cambiar su ruta si el mensaje advierte sobre un accidente o congestión en la vía.
- **handleSelfMsg:** es un método de gestión de eventos internos que funciona como un sistema de temporizadores. En lugar de recibir mensajes de otros, este método procesa mensajes que el propio vehículo se envió a sí mismo para el futuro mediante una programación de tiempo. Es fundamental para manejar retrasos intencionados, enviar mensajes periódicos (beacons) o ejecutar acciones diferidas sin detener el reloj global de la simulación. En el simulador Veins (que combina OMNeT++ y SUMO), estas tres funciones son pilares fundamentales para definir cómo se comporta un vehículo en una red vehicular (VANET).
- **handlePositionUpdate:** esta es una función de sincronización entre el tráfico y la red. Se llama cada vez que el vehículo se mueve en el simulador de tráfico (SUMO). Sirve para que la parte de red (OMNeT++) sepa dónde está el coche físicamente. Este es el lugar ideal si quiere que el vehículo tome una decisión basada en su ubicación actual (por ejemplo: "Si ya pasé la intersección X, deja de enviar la alerta"). Algunos ejemplos de funciones de TraCI en Veins se listan en la Tabla 11.3.

11.3.3 Parte III: Ajuste la ubicación del RSU (access point) de su mapa de Veins

1. En el archivo `omnetpp.ini` se define la red mediante el parámetro `network = RSUExampleScenario`. Acceda al archivo `RSUExampleScenario.ned` y configure en 0 el número de *Roadside Units* (RSUs). Ejecute nuevamente la simulación para verificar que la unidad de infraestructura ya no aparece en el escenario.

<code>rsu [0]</code>

2. Restablezca la presencia de la unidad en el escenario de simulación. Para ello, diríjase nuevamente a `RSUExampleScenario.ned` y habilite una instancia de la RSU:

Table 11.3: Ejemplos de comandos y variables de TraCI en Veins 5.3

Componente / Tipo	Elemento	Descripción / Propósito
Control de Ruta	<code>changeRoute()</code>	Modifica la trayectoria basada en datos de tráfico.
	<code>changeTarget()</code>	Cambia el destino final del vehículo dinámicamente.
Control de Movimiento	<code>setSpeed()</code>	Establece una velocidad específica en m/s.
	<code>slowDown()</code>	Reduce la velocidad gradualmente en un tiempo dado.
	<code>setStop()</code>	Obliga al vehículo a detenerse en una posición exacta.
Estado del Vehículo	<code>getSpeed()</code>	Obtiene la velocidad actual del nodo en SUMO.
	<code>getRoadId()</code>	Devuelve el identificador de la calle (<i>edge</i>) actual.
	<code>setColor()</code>	Cambia el color del vehículo en la interfaz de SUMO.
Variables de Datos	<code>coordX, coordY</code>	Coordenadas geográficas (X, Y) del nodo origen.
	<code>currentSpeed</code>	Variable que almacena la velocidad actual del vehículo.
	<code>messageCount</code>	Contador secuencial para la gestión de mensajes enviados.
	<code>ttl</code>	<i>Time to Live</i> : control del límite de saltos en la red.

```
rsu [ 1 ]
```

3. Modifique la ubicación de la RSU dentro del archivo `omnetpp.ini` para posicionarla estratégicamente en su mapa utilizando coordenadas cartesianas:

```
*.rsu [ 0 ]. mobility . x = 2000
*.rsu [ 0 ]. mobility . y = 2000
*.rsu [ 0 ]. mobility . z = 3
```

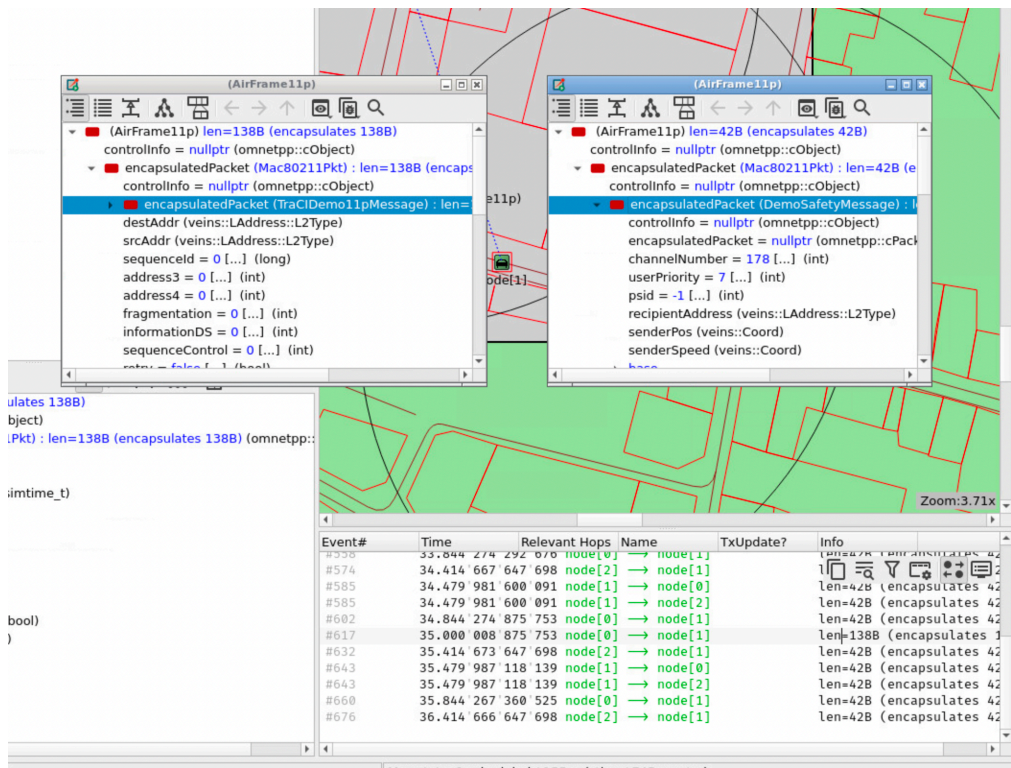


Figure 11.6: Tipos de mensajes en comunicaciones V2V

11.3.4 Parte IV: Reto de comunicaciones V2V

1. **Configure su escenario de prueba.** Incluya su propio mapa (zona pequeña) y dos vehículos cercanos entre ellos (referirse a la Parte I). Modifique el archivo `omnetpp.ini` para delimitar el radio de comunicación y habilitar la visualización de interferencia:

```
*.connectionManager.maxInterfDist = 100m
*.connectionManager.drawMaxIntfDist = true
```

2. Implemente su escenario de pruebas y **analice la lógica de difusión de alertas por accidente en el escenario base de Veins**. Recuerde que el funcionamiento del escenario de ejemplo o demo de Veins se detalla en el archivo `TraCIDemo11p.cc` ubicado en el proyecto de `veins-veins-5.3.1/src/veins/modules/application/traci/TraCIDemo11p.cc` (referirse a la Parte II). Basándose en los métodos de la Tabla 11.2, responda:
 - **Lógica de generación:** Qué evento dispara la transmisión del mensaje?, Son mensajes periódicos o por evento único?
 - **Comportamiento del Receptor:** Al recibir un mensaje, qué acciones toma el vehículo (frenado, cambio de ruta, retransmisión)?
 - **Lógica de Colores:** Explique la condición configurada para que los vehículos cambien su color a **rojo** (alerta/accidente) o **verde** (recepción exitosa de datos).
3. **Modifique campos del mensaje (`TraCIDemo11pMessage.msg`).** El escenario por defecto de Veins usa el mensaje `TraCIDemo11pMessage.msg` ubicado en `veins-veins-5.3.1/src/veins/modules/application/traci/TraCIDemo11pMessage.msg`. Extienda la estructura del mensaje `TraCIDemo11pMessage.msg` para incluir telemetría y control de red. Agregue los siguientes campos:
 - `double coordX, coordY`: Coordenadas del nodo origen.
 - `double currentSpeed`: Velocidad actual del vehículo.
 - `int ttl`: Time to Live para control de saltos.

Algunos ejemplos de interés de TraCI, con los que puede actualizar los valores de los campos del mensaje, se resumen en la Tabla 11.3.

4. Ejecute nuevamente la simulación e **identifique los diferentes mensajes que se intercambian entre vehículos**. Utilice el inspector de objetos de OMNeT++ (clic derecho sobre el *Airframe*) para analizar el intercambio de mensajes entre vehículos. Explique las diferencias funcionales de los mensaje: (i) **DemoSafetyMessage (DSM)**: mensajes básicos de seguridad, y (ii) **TraCIDemo11pMessage**: mensaje de aplicación modificado con los nuevos campos.
5. Modifique el archivo de rutas de SUMO (`.rou.xml`) para configurar un flujo constante de tráfico (aproximadamente 5 vehículos) operando sobre una misma trayectoria. Tras ejecutar la simulación, analice el alcance de la comunicación y determine el número máximo de saltos (*hops*) que alcanza el mensaje de la aplicación demo de Veins de forma predeterminada.
6. Implemente un mecanismo de control de propagación multihop limitando el reenvío a un máximo de 4 saltos. Para ello, utilice el campo `ttl` (Time to Live) definido previamente en el paso 3, asegurando que el mensaje sea descartado una vez alcanzado dicho umbral.